
DPGrammar: Joint Viterbi Repair for Grammar-Constrained Diffusion Language Models

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Constrained decoding keeps LLM output in a format like JSON by masking invalid
2 tokens with a left-to-right parser. Diffusion language models unmask by confidence
3 rather than in order, so a token is final before the parser reaches it: the grammar
4 cannot be enforced ahead of the parse frontier, only repaired behind it. One position
5 at a time is not enough, because a grammar couples the positions it constrains
6 and a locally probable fix can eliminate every valid completion of the region.
7 **DPGrammar** decides the violated span as a whole. Merging paths that leave
8 the parser in the same state, it runs a Viterbi pass over (parser state $\times$ position)
9 pairs and returns in polynomial time the likeliest valid assignment in the lattice it
10 expands, with candidates drawn from the automaton’s outgoing edges rather than
11 the model’s ranking, so a tighter budget costs likelihood and never validity. On
12 JSONSchemaBench medium with LLaDA-8B-Instruct, schema validity rises from
13 91.8% to 96.7% and corrective work falls from 33.2 to 1.2 remasks per instance,
14 for 3.3 s of added mean latency spent writing $1.7\times$ the tokens. The margin over
15 the unrepaired baseline more than doubles, to 10.3 pp, once outputs must carry
16 content, and reaches 13.4 pp on the hardest split.

17 1 Introduction

18 Masked diffusion language models (dLLMs) such as LLaDA-8B [Nie et al., 2025] and Dream-7B
19 [Ye et al., 2025] generate by iteratively unmasking positions in confidence order rather than left to
20 right, decoding many tokens per forward pass and trading sequential depth for parallelism. Many
21 applications need that output to be structured: an agent calling a tool needs JSON conforming to a
22 given schema [Patil et al., 2025, Geng et al., 2025], and one misplaced brace makes the response
23 unusable to the caller no matter how good its content. Autoregressive (AR) models meet this
24 requirement through constrained decoding: an incremental parser tracks the partial output and masks
25 any next token that would violate the grammar [Willard and Louf, 2023], machinery now standard in
26 serving frameworks [Dong et al., 2024, Guidance AI, 2023, Zheng et al., 2024, Microsoft, 2025].

27 That machinery assumes left-to-right commitment, whereas a dLLM commits out of order: under
28 carry-over unmasking [Sahoo et al., 2024, Shi et al., 2024], once a position is revealed it remains
29 fixed, so the parser may eventually reach an earlier token it rejects but can no longer modify. One
30 answer is to prevent violations outright, making the constraint globally tractable and sampling from
31 the constrained posterior exactly [Suresh et al., 2025]. That guarantees satisfaction by construction,
32 but it needs a regular representation, and a recursive JSON Schema with unbounded nesting is not

regular.¹ It also pays at every denoising step and fixes the decoding order, where repair costs nothing on a clean step and leaves the schedule free. We therefore repair after the fact, forgoing the certified guarantee. Existing repair, though, is myopic [Zhang et al., 2026, Mündler et al., 2025a]: it rewrites the position the parser rejected and leaves the rest of the region alone. That is not enough, because whether a token leads anywhere depends on what must follow it: a choice that is both locally probable and locally legal can still eliminate every valid completion of the region. The model gives no signal about this, its per-position distributions being independent where the grammar is not.

We introduce **DPGrammar**, a repair layer that treats a violated region as a single decision problem rather than a sequence of independent ones. When the parser rejects at the frontier, we identify the affected span and run a Viterbi search over a lattice of (parser state $\times$ position) pairs. Deciding an affected region jointly would be exponential in span length if searched over raw token sequences. However, because a path’s future valid continuations are determined by its current parser state rather than its prefix, paths arriving at identical parser states can merge into a single search node, reducing the Viterbi pass to time linear in span length. We key states on the token set the parser admits, which merges more aggressively than that argument licenses; §3.3 measures what the approximation costs. At each node, candidate tokens are drawn from the parser automaton’s outgoing edges and ranked by model likelihood, which narrows the search while guaranteeing that no legal token is missed, so a tighter budget costs likelihood and never validity.² Termination is likewise read off the grammar rather than a step count; the layer places no requirement on the unmasking schedule, since it fires only after a rejection; and delegating the parse to an off-the-shelf incremental parser [Microsoft, 2025] makes the constraint context-free rather than regular.

We evaluate on JSONSchemaBench [Geng et al., 2025] and JSON-Mode-Eval [Nous Research, 2024, ETH SRI, 2025] with LLaDA-8B-Instruct, against published constrained decoders and against the same system with the repair layer disabled. On 511 medium instances, DPGrammar reaches 96.7% schema validity against 91.8% with the layer off, and cuts mean repair events from 33.2 to 1.2. The layer’s benefit is not a constant offset but grows with the density of joint constraints, from +0.2 to +13.4 pp of validity between the easiest and hardest split. On a grammar whose violations are purely local the layer never fires at all, which places the same claim at the other end of that axis.

2 Related Work

Constrained decoding for AR LLMs. Constrained decoding for AR LLMs compiles grammars into incremental automata that mask invalid logits at each step [Willard and Louf, 2023, Dong et al., 2024, Microsoft, 2025], with much recent work spent on making that mask cheap [Ugare et al., 2025, Park et al., 2025] or on extending it to semantic [Poesia et al., 2022] and type-system [Mündler et al., 2025b] constraints. Geng et al. [2023] give the framing we adopt, that a formal grammar describes the output space of structured NLP tasks in general rather than of serialisation formats alone, and Tuccio et al. [2025] carry it into open-ended generation; our constraint is context-free for the same reason, although we evaluate on JSON Schema. All of it assumes monotonic left-to-right generation, so out-of-order commitment never arises, and that is the central problem in dLLMs.

Constrained decoding for dLLMs. Mündler et al. [2025a] propose a CFG-constrained dLLM decoder, intersecting the grammar with an automaton over the generated prefix. DINGO [Suresh et al., 2025] performs MAP inference over a deterministic finite automaton weighted by the model’s mean-field predictions, and Dang and Ermon [2026] generalise this to exact sampling from the constrained mean-field posterior under arbitrary remasking schedules, restoring a hard guarantee. Both require the constraint as a finite automaton, which bounds them to regular languages; we trade that guarantee for a context-free constraint language and a schedule-agnostic layer. Cardei et al. [2025] likewise prevent rather than repair, but through a differentiable projection applied inside the sampling loop rather than through an automaton, which reaches constraints that admit a continuous relaxation, such as lexical inclusion or molecular validity; grammar membership is a discrete predicate over the token sequence and is the case a parser handles directly. LAVE [Zhang et al., 2026] verifies each frontier token by sampling K random completions, accepting only if one is grammar-valid; it is the strongest published baseline we are able to run, and the diffusion member of the lookahead-then-verify family,

¹On JSONSchemaBench medium an automata toolchain compiles only 383 of 586 schemas where an incremental context-free parser handles 511 (Appendix G).

²Against a vocabulary of 126,349, a live parser state admits a median of just 106 tokens (§3.4, Appendix A).

whose K roll-outs per token Nakshatri et al. [2025] make cheaper in the AR setting by drafting them with a smaller model. Plan-style decoders [Li et al., 2026] emphasise structure-aware planning. In contrast, we maintain a lightweight incremental parser at the token level, triggering joint search where parsing conflicts arise.

Viterbi decoding for non-AR models. Lattice search is well established in non-autoregressive generation [Deng and Rush, 2021]: Shao et al. [2022] apply Viterbi decoding over Directed Acyclic Transformers and Control-DAG [Chen et al., 2024] enforces lexical constraints with weighted finite-state automata. Both search a model-defined lattice in which output length must be constrained, or an unnormalised likelihood favours trivially short paths. Ours is defined by the parser and bounded by the violated span, so length is fixed rather than searched and the length-control term those methods need has nothing to correct, which §4.4 measures.

3 Method

3.1 Problem statement

Let G be a grammar and let its *incremental parser* be an object that processes tokens left to right. After accepting a prefix, the parser maintains a *state* q defining the valid continuations permitted by G . Write $\mathcal{L}(q) \subseteq V$ for the set of tokens the parser can consume from q , its outgoing edge set (Appendix E shows the concrete interface). The decoder holds a sequence x over $V \cup \{\text{MASK}\}$; its frontier c is the length of the longest prefix the parser has consumed, and q_c the state it reached there. Because unmasking follows confidence rather than position, the region beyond the frontier mixes masked slots with tokens the model has already placed:

$$\underbrace{x_0 \cdots x_{c-1}}_{\text{consumed, parser at } q_c} \quad \bigg| \quad \underbrace{x_c}_{\text{frontier}} \quad \bigg| \quad \underbrace{\text{MASK } x_{c+2} \text{ MASK } x_{c+4} \cdots}_{\text{placed but never parsed}}$$

Only x_c is under the parser’s control: frontier masking guarantees $x_c \in \mathcal{L}(q_c)$ when x_c is chosen this step, while every position to its right was sampled with no constraint at all.

Those positions are not provisional. Masked diffusion is *carry-over unmasking* [Sahoo et al., 2024, Shi et al., 2024]: the reverse posterior copies any already-revealed position unchanged, so a step only ever replaces MASK symbols,

$$x_0 \leftarrow \mathbf{1}[\text{mask}] \odot x_0 + (1 - \mathbf{1}[\text{mask}]) \odot x,$$

which is what LLaDA’s reference sampler implements. (Samplers that deliberately remask revealed positions relax the property; we do not assume one.) Tokens placed beyond the frontier are therefore immutable commitments. As the remaining masks fill in, the parser advances across the newly contiguous region and may reach a position $v > c$ where $x_v \notin \mathcal{L}(q_v)$: a *violation*, a final token the grammar refuses.

Why this is challenging. Every constrained decoder for an autoregressive model is a filter: it inspects one position’s distribution and removes what the grammar forbids. A violator sits past the frontier, where nothing can be filtered, only overwritten. Local repair rarely suffices, because the tokens that interact to cause the violation are often several positions away. The problem is a *search* over joint assignments to a region, not a filter over one distribution.

3.2 Repair as bounded search

Given a violator v , the layer repairs a bounded span $[v, e)$: it rewrites committed positions and never fills holes. Three things bound e . A forward scan from v reads the tokens already in place and stops at the first one the grammar accepts *at bracket depth zero*, which is the span end when it finds one; failing that it stops at a compute budget of $L = 48$ positions. The span is then truncated at the first MASK, since a hole is the sampler’s to fill and not ours. The depth test is the part that matters: without it the span can end inside an unclosed array or object, where the cheapest legal continuation is the closing bracket, and the joint solve then collapses the rest of the structure to $[]$ or $\{\}$, the degeneracy §4.2 measures with the content floor. Appendix A reports which of the three binds in practice, and how long the resulting spans are.

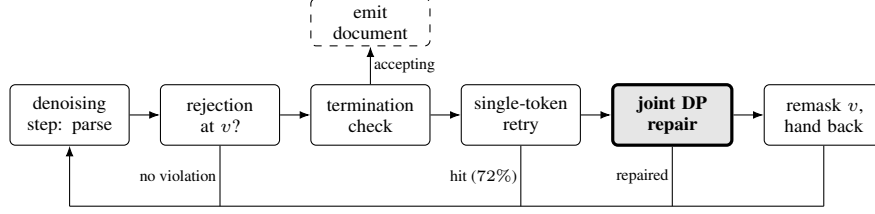


Figure 1: Where the joint repair fires. The row is tried left to right and each stage is reached only when the one before it did not settle the position; every outcome but emission returns to the next denoising step. The shaded box is this paper.

Over that span we seek the assignment y maximising $\sum_p \log p_\theta(y_p \mid x)$ subject to the parser accepting $y_v \cdots y_{e-1}$ from q_v . §3.3 solves it as a Viterbi pass and §3.4 says where the candidates at each position come from.

A rejection at v enters the cascade of Figure 1. Termination is tested first: EOS is not consumable, so a finished document reaches the frontier looking exactly like a violation. We separate the two on the grammar rather than on a step budget. The harness ends the document only when $\mathcal{L}(q) \setminus \{\text{EOS}, \text{EOT}\} = \emptyset$ at an accepting q , where ending is not a choice anyone is making; the model ends it when it places a stop token at the frontier and q is accepting, where the model has asked and the grammar need only permit. Next is a bounded single-token retry, which walks the model’s ranked logits at v for up to 10 candidates and takes the first the parser accepts. That is filter-then-propose, the pattern §3.4 argues against, but a miss here is free because the position falls through to the joint search, and it resolves 72% of violations at negligible cost. Only then is the span solved jointly (§3.3); if no assignment exists, v is remasked for the sampler to re-derive.

3.3 Joint repair by Viterbi over parser states

Algorithm 1 gives the search. It is a Viterbi pass over layers indexed by position, where a node is a reachable *parser state* rather than a token history. Candidates that leave the parser in the same state κ' collapse to the single highest-scoring path (line 11), and that merge is what bounds the search. Enumerating assignments over the span keeps $O(k^{\text{span}})$ hypotheses alive; merging caps each layer at the number of distinct keys $|S|$, giving $O(\text{span} \cdot |S| \cdot k)$ parser operations and $O(|S|)$ live parsers. The key $\kappa(q) = \text{bytes}(\mathcal{L}(q))$ is the parser’s own mask, which line 5 already computed to build C , so the merge costs no parser call of its own. Since $|S|$ stays in the single digits in practice, span length stops being an exponent in either time or memory.

What merging costs. κ is a state *abstraction*, not a state identifier: two states can admit the same tokens and still differ in what the grammar demands later, so merging on it is not exact. It is nonetheless sound, because the key prunes but never reconstructs. Each retained node holds the parser it reached and the backtrace follows predecessor links, so what the pass returns is always a sequence some parser consumed from q_v , and a collision cannot produce a span the grammar rejects. What a collision can cost is the objective: the lower-scoring of two genuinely different states is dropped, and where the survivor dead-ends first the span ends early, which Algorithm 1 treats as finishing rather than failing. Appendix E measures that cost by widening the merge, and finds it changes the returned assignment at 0.33% of solves.

3.4 Candidates from the automaton

At each lattice node the search needs a set of tokens to try. We draw it from the parser rather than from the model: for a live state q we read $\mathcal{L}(q)$, the tokens that state can consume, and rank *within* it by model likelihood,

$$\mathcal{C}(q, p) = \text{top-}k_{t \in \mathcal{L}(q) \setminus \{\text{EOS}\}} \log p_\theta(t \mid x, p). \quad (1)$$

The alternative is to take the model’s top- k predictions over V and retain only those the parser accepts. Filtering after sampling has a structural failure mode: $|\mathcal{L}(q)|$ is set by the grammar rather than by a hyperparameter, so no choice of k removes the risk: where the admitted set is small the draw omits it entirely and the DP reports “no path” at a position where one exists (Appendix A works one through).

Algorithm 1 Joint repair of a violated span

Require: violator v , span end e , parser state q_v , log-probabilities $\log p_\theta$, candidate budget k
Ensure: an assignment over $[v, e']$, $e' \leq e$, that the parser accepts from q_v

```
1:  $S \leftarrow \{ \kappa(q_v) \mapsto (q_v, 0) \}$  ▷ state key  $\mapsto$  (parser, score)
2: for  $p = v$  to  $e - 1$  do
3:    $W \leftarrow \emptyset$  ▷ best incoming path per successor state
4:   for all  $(q, s) \in S$  do
5:      $C \leftarrow$  the  $k$  most probable tokens of  $\mathcal{L}(q) \setminus \{\text{EOS}\}$  ▷ §3.4
6:      $C \leftarrow C \cup \{x_p\}$  if  $x_p \in \mathcal{L}(q)$  ▷ keep the model's own token reachable
7:     for all  $t \in C$  do
8:       advance  $q$  by  $t$ ;  $\kappa' \leftarrow \kappa(q)$ ;  $s' \leftarrow s + \log p_\theta(t)_p$ 
9:       rollback  $q$  one token ▷ probe without copying
10:      if  $\kappa' \notin W$  or  $s' > \text{score}(W[\kappa'])$  then
11:         $W[\kappa'] \leftarrow (q, t, s')$  ▷ merge: paths reaching  $\kappa'$  collapse to the best one
12:      end if
13:    end for
14:  end for
15:  if  $W = \emptyset$  then
16:    break ▷ nothing consumable: the span ends here, it does not fail
17:  end if
18:   $S \leftarrow \{ \kappa' \mapsto (\text{clone}(q) \text{ advanced by } t, s') : (\kappa', (q, t, s')) \in W \}$ 
19: end for
20: return the backtrace from  $\arg \max_{q \in S} \text{score}(q)$ 
```

168 Drawing from $\mathcal{L}(q)$ removes that failure mode by construction, because the proposal *is* the accepted
169 set. What k bounds is therefore likelihood, not validity. Validity is unconditional: every candidate
170 comes from $\mathcal{L}(q)$, so no k , however small, lets the search emit an invalid span. An answer is always
171 returned, since no live state we measured has an empty legal set, and C is empty only where the
172 grammar admits nothing but a stop token, in which case the document is finished and there is nothing
173 to place. Only optimality is relative to the lattice: the pass returns the best assignment the candidate
174 sets span, which is the best the grammar accepts exactly where those sets are complete, and where
175 $|\mathcal{L}(q)| \leq k$ the edge set is enumerated rather than sampled. Appendix A gives the distribution of
176 $|\mathcal{L}(q)|$, how often it fits inside k , and where the span budget L binds.

177 4 Experiments

178 4.1 Setup and metrics

179 **Datasets.** JSONSchemaBench [Geng et al., 2025] supplies JSON schemas without reference
180 answers, in three difficulty splits: *easy* and *medium* (586 instances each) and *hard* (368). Schemas
181 that LLGUIDANCE cannot compile are excluded from every arm, since no method runs on them: 28,
182 75, and 99 respectively, leaving $n = 558, 511, 269$. JSON-Mode-Eval supplies prompts paired with
183 a schema *and* a reference completion; we use the 272-instance extension [ETH SRI, 2025] of the
184 original 100-row release [Nous Research, 2024], as released with CD-CFG.

185 **Models.** All arms run LLaDA-8B-Instruct [Nie et al., 2025] with $T=128$ denoising steps and block
186 length 32, on a single A100 per shard, with grammars compiled by llguidance [Microsoft, 2025]
187 except where noted. Appendix D lists the remaining constants.

188 **Baselines.** *Vanilla* is the same decoder with no constraint of any kind, and marks what the backbone
189 produces unaided. *CD-CFG* [Mündler et al., 2025a] is run from the authors' released implementation,
190 which builds its constraint with RUSTFORMLANG, the formal-language library that ships with it; §G
191 treats the resulting coverage difference separately. *LAVE* [Zhang et al., 2026] is likewise run from
192 its authors' released code, instrumented only to record per-operation timing, under a 120 second
193 per-instance cap that applies to every arm and scores an instance reaching it as a failure. *no-DP* is
194 our own system with the repair layer switched off: *the same binary, the same checkpoint, the same*
195 *schedule and the same seed*. Upon a violation, it retains a bounded single-token retry and, if that is

Table 1: The difficulty gradient on LLaDA-8B-Instruct. Each arm is scored on the instances its own toolchain compiles, so n differs: LAVE, no-DP and DPG share the LLGUIDANCE set exactly, while $\dagger$ marks CD-CFG, whose RUSTFORMLANG front-end accepts a smaller subset (Appendix G). Times are in seconds and TO counts the 120 s cap. rs/inst is corrective work per instance, whose unit differs by arm (remasks, lookahead draws, rejected candidates), so only no-DP and DPG compare there. DPG is DPGrammar.

Split	Method	n	schema@1 $\uparrow$	content $\uparrow$	mean $\downarrow$	p50 $\downarrow$	p95 $\downarrow$	p99 $\downarrow$	TO $\downarrow$	rs/inst $\downarrow$
easy	Vanilla	577	62.9	30.2	8.34	8.37	11.23	14.98	0	—
	CD-CFG $\dagger$	449	56.3	28.1	6.22	6.72	11.54	15.98	0	49.2
	LAVE	558	80.1	40.5	30.63	6.71	120.01	120.01	94	207.3
	no-DP (ours)	558	97.5	45.2	7.58	7.58	11.47	15.09	0	20.7
	DPG (ours)	558	97.7	49.3	5.31	3.40	15.11	29.06	0	0.6
medium	Vanilla	586	49.8	47.1	14.89	14.36	21.82	31.67	0	—
	CD-CFG $\dagger$	383	44.1	41.0	22.71	14.16	112.24	120.00	19	57.3
	LAVE	511	78.3	74.0	38.50	25.70	120.00	120.01	60	253.5
	no-DP (ours)	511	91.8	74.6	13.45	13.40	21.43	29.99	0	33.2
	DPG (ours)	511	96.7	84.9	16.75	15.05	36.21	63.24	0	1.2
hard	Vanilla	368	20.7	20.4	44.64	35.43	103.48	120.00	4	—
	CD-CFG $\dagger$	245	11.4	10.6	57.61	41.53	120.00	120.15	72	50.2
	LAVE	269	39.8	39.4	40.47	38.35	120.00	120.00	30	115.8
	no-DP (ours)	269	74.3	53.9	31.41	26.69	61.91	101.51	0	56.0
	DPG (ours)	269	87.7	71.7	42.66	35.24	96.58	115.96	2	1.5

exhausted, defers the position back to the sampler. Both arms share the system-level optimisations that keep the decoder fast, which Appendix C describes. The closest work we do *not* run is DINGO [Suresh et al., 2025] and the exact sampling of Dang and Ermon [2026]: both need the constraint as a finite automaton, which the recursive schemas here do not admit, and neither had a released implementation at the time of writing. §4.3 reports what that guarantee would have traded, measured with our own implementation of exact constrained inference.

Metrics. Three quality metrics. *schema@1* is validation against the instance’s own schema under *jsonschema*. *content* additionally requires five scalar leaves, since validity alone cannot separate an empty structure from a populated one; Table 2 sweeps the threshold and Appendix J shows a pair the two order differently. *functional@1* requires a match against the reference completion, so it is JSON-Mode-Eval only. Two cost metrics: corrective work per instance (*rs/inst*, defined with Table 1) and wall-clock latency, reported as mean, p50, p95 and p99 with timeouts at a 120 s cap.

4.2 Main results

Table 2: JSONSchemaBench *medium* ($n=511$) as the content floor rises. Each row requires validity *and* at least κ populated leaves.

	LAVE	no-DP	DPG rammar	Δ vs no-DP	Δ vs LAVE
validity only	78.3	91.8	96.7	+4.9	+18.4
$\wedge \kappa=1$	78.3	84.9	93.7	+8.8	+15.4
$\wedge \kappa=3$	76.7	79.1	90.6	+11.5	+13.9
$\wedge \kappa=5$	74.0	74.6	84.9	+10.3	+10.9
$\wedge \kappa=10$	56.0	50.7	60.5	+9.8	+4.5
$\wedge \kappa=15$	33.7	29.5	35.0	+5.5	+1.3

Repair pays in proportion to how jointly the grammar constrains a region, and schema validity hides most of what it pays. On *medium* the layer lifts validity by 4.9 pp over the unrepaired baseline while cutting corrective work by $28\times$, from 33.2 to 1.2 remasks per instance. Two trends say more than that headline does. Raising the content floor doubles the margin, from +4.9 to +10.3 pp at five populated leaves and +11.5 at three (Table 2), because single-token repair takes the cheapest legal token at a violation and a closing bracket usually is one: the document collapses exactly where the

joint solve would have continued it. Raising the difficulty does the same and monotonically, by +0.2, +4.9 and +13.4 pp of validity and +4.1, +10.3 and +17.8 pp of content across *easy*, *medium* and *hard*, which is what joint repair predicts: violations become more joint as schemas acquire nesting, required-property ordering and cross-field constraints. Against LAVE the shape inverts, narrowing from +18.4 to +1.3 pp as the floor rises, because LAVE never rewrites and its surviving documents were never at risk of collapsing. Against LAVE we buy frequency; against the unrepaired baseline we buy content (Appendix J).

That margin is not bought with time, and it does not depend on which instances each arm is scored on. The constraint machinery costs what no-DP’s does, 909 against 920 ms, so the 3.3 s is not the search but the decoding the repair makes possible: a repaired document keeps going where no-DP stops, and per token emitted the cost rises by 15% where the total rises by 96% (Appendix I). That brevity is premature termination rather than economy, and the content floor scores it as the failure it is: 18.8% of no-DP’s valid *medium* outputs carry fewer than five leaves against 12.1% of ours. Where repair is unnecessary the ordering reverses outright, and on *easy* DPGrammar is faster in both mean and median. The tail does grow, from 21.4 to 36.2 s at p95, and two *hard* instances reach the cap where no-DP reaches none. Restricting each comparison to a shared population leaves the ordering unchanged, whether that population is the 351 schemas both toolchains compile or the 451 instances LAVE finishes within the cap (Appendix G).

The content margin survives a change of schedule and grows where the schedule is harder: +13.1 pp when the whole sequence is committed in one block rather than in blocks of 32, and +17.2 when positions are revealed in random order rather than by confidence, against +10.4 here. The validity margin travels less evenly, from +15.1 under the first to +0.4 under the second, and what it tracks is the step budget rather than the search. Random order produces 2.3× the violations, repairing them costs denoising steps, and 46% of instances then exhaust the 128-step schedule before the document closes, which is 98% of what DPGrammar fails on there; the search itself dead-ends once in 733 calls (Appendix H).

Not every residual failure is a failure of the search. LLGUIDANCE pins an object’s properties to their declaration order, so a model that writes them in another order produces a violation *jsonschema* would not call an error at all, and whose correct repair is a movement rather than a substitution, which a positional DP cannot express. This accounts for 6.6% of the 1,106 violations the reported run records, and for 22% of the 120 that arise where the grammar admits a single token, the states in which the search has no alternative to write (Appendix B). It is a limit of overwriting in place rather than of solving the span jointly.

4.3 Two comparisons the main table cannot make

Table 3: JSON-Mode-Eval, $n=272$, LLaDA-8B-Instruct. *schema@1* is the same quantity as in Table 1; *functional@1* additionally requires the output to match the reference completion.

	Vanilla	CD-CFG	LAVE	no-DP	DPGrammar
<i>schema@1</i> ↑	85.3	92.6	98.2	97.4	98.2
<i>functional@1</i> ↑	48.5	52.6	53.7	53.7	52.6

Constraining structure does not buy semantics. JSON-Mode-Eval is where the two separate, since it pairs each prompt with a reference completion: every constrained arm gains 7 to 13 pp of schema validity over Vanilla while *functional@1* stays inside a 5 pp band. The layer itself has little to do here, because no-DP already validates 97.4% of these instances, so the joint search fires on 14 of 272 and the two arms agree byte for byte on 256. What separates their *functional@1* is five instances, one in our favour and four against, which an exact McNemar test places inside noise ($p = 0.38$). The 1.1 pp gap measures this sample rather than what rewriting costs an answer; §4.2 is where the layer runs under joint-constraint load.

What prevention would have cost. Prevention buys its guarantee three times over. Run on the same backbone and schedule, exact inference over the constrained posterior reaches a third fewer schemas than the incremental parser, is three to five times slower, and returns a median of one scalar leaf against our thirteen. That last is the content floor again, from the other side: the smallest

document a schema admits is both valid and cheap, so a decoder optimising probability under a hard constraint is drawn to it (Appendix F).

4.4 Ablations: what moves the metric is the search space, not the objective

Table 4: Ablation of the repair layer’s components, on JSONSchemaBench medium with one seed. The upper block changes what the search ranges over; the merge width is measured on the search itself rather than end to end (§3.3). The lower block changes the objective or the admission rule and leaves schema@1 where it is.

Component	Varied	Measured effect
Candidate source	vocabulary top- k $\rightarrow$ automaton	schema@1 95.5 $\rightarrow$ 96.7; dead ends 23.3 $\rightarrow$ 1.0%
Repair window	single token $\rightarrow$ full span	1-char leaves 12.3% $\rightarrow$ 7.9%
Paths kept per key	1 $\rightarrow$ 4	assignment changes at 0.33% of solves
Objective	$\log p \rightarrow$ min-edit	−0.5 edits, 93.9% identical
Candidate admission	always admit committed token	none (identical output)
Stop tokens	excluded $\rightarrow$ kept in $\mathcal{C}$	99.7% identical; 48 $\times$ futile probes
Post-hoc enrichment	on $\rightarrow$ off	none (identical output)
Edit penalty	$\lambda \in \{0, 2\}$	none (never non-zero)

We vary one component at a time, holding backbone, seed and schedule fixed, so what follows is evidence about this configuration rather than a law. Three effects the table cannot show are worth stating. Filtering the vocabulary after sampling abandons 273 positions that the edge set recovers, and the three dead ends the edge set does produce are not proposal misses: they are states admitting nothing but a stop token, the one case $\mathcal{C}$ is empty by design. Minimum-edit attributes 90% of what the DP rewrites to the grammar rather than to the objective. And the edit penalty stays at zero although it is the correction Viterbi decoders for non-autoregressive models need to stop an unnormalised objective collapsing toward short paths [Shao et al., 2022, Chen et al., 2024]: our span is fixed by the violation rather than chosen by the search, so there is no collapse to prevent.

5 Conclusion

We present DPGrammar, a repair layer for grammar-constrained decoding in diffusion language models, which commit tokens out of order and can therefore only repair a violation after the fact. Casting that repair as a dynamic program over parser states lets a violated region be decided as a whole rather than one position at a time, with paths that leave the parser in the same state merging into one. Because candidates are read off the parser rather than the vocabulary, bounding that search trades likelihood and never validity. What that buys is not a constant offset. The margin over repairing one position at a time doubles once outputs must carry content, and grows monotonically with how jointly a schema constrains the region it fails in, which is where prevention pays instead: in coverage, in latency, and in documents that satisfy a schema by saying nothing. Validity is held at the smallest intervention the grammar allows, by a search that fires only after a rejection, proposes only tokens the parser admits, follows the model’s ranking wherever the grammar leaves a choice, and overwrites a position only where it does not.

6 Limitations and future work

Our results are one backbone, one seed and one constraint family, and the last of those bounds more than the numbers: a constraint whose states are harder to tell apart by their legal-token sets would pay more for the state key than the 0.33% we measure here (§3.3). The layer also earns nothing where violations are local rather than joint, and on SMILES the joint search seldom fires at all; that is the difficulty gradient read at its other end, but it leaves the boundary between those grammars and JSON Schema unmapped, and other backbones and constraint families would place it.

A more fundamental direction is the parser. Ours is written for left-to-right decoding, which is why the constraint can be enforced at one frontier and everything past it only repaired. A checker that tracked several disconnected regions at once would let the grammar be enforced wherever a step touches, and give a joint repair more of the sequence to work with.

References

- Michael Cardei, Jacob K. Christopher, Thomas Hartvigsen, Bhavya Kailkhura, and Ferdinando Fioretto. Constrained discrete diffusion. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2503.09790.
- Jinghong Chen, Weizhe Lin, Jingbiao Mei, and Bill Byrne. Control-DAG: Constrained decoding for non-autoregressive directed acyclic T5 using weighted finite state automata. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Short Papers)*, 2024.
- Meihua Dang and Stefano Ermon. Constrained decoding for diffusion language models via efficient inference over finite automata. *arXiv preprint arXiv:2607.07026*, 2026.
- Yuntian Deng and Alexander M. Rush. Sequence-to-lattice models for fast translation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, Punta Cana, Dominican Republic, 2021. URL <https://aclanthology.org/2021.findings-emnlp.318/>.
- Yixin Dong, Charlie F Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. Xgrammar: Flexible and efficient structured generation engine for large language models. *Proceedings of Machine Learning and Systems* 7, 2024.
- ETH SRI. json-mode-eval-extended. Hugging Face dataset, 2025. URL <https://huggingface.co/datasets/eth-sri/json-mode-eval-extended>. Extension of NousResearch/json-mode-eval to 272 instances.
- Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952, Singapore, 2023. URL <https://aclanthology.org/2023.emnlp-main.674/>.
- Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. Jonschemabench: A rigorous benchmark of structured outputs for language models, 2025. URL <https://arxiv.org/abs/2501.10868>.
- Guidance AI. Guidance: A guidance language for controlling large language models, 2023. URL <https://github.com/guidance-ai/guidance>.
- Miao Li, Hanyang Jiang, Sikai Cheng, Hengyu Fu, Yuhang Cai, Baihe Huang, Tinghan Ye, Xuanzhou Chen, and Pascal Van Hentenryck. Plan, verify and fill: A structured parallel decoding approach for diffusion language models, 2026. URL <https://arxiv.org/abs/2601.12247>.
- Microsoft. LLMGuidance: Fast structured outputs for LLMs. GitHub repository, 2025. URL <https://github.com/guidance-ai/llguidance>.
- Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, and Martin Vechev. Type-constrained code generation with language models. *Proceedings of the ACM on Programming Languages*, 9(PLDI), 2025b. doi: 10.1145/3729274.
- Niels Mündler, Jasper Dekoninck, and Martin Vechev. Constrained decoding of diffusion llms with context-free grammars, 2025a. URL <https://arxiv.org/abs/2508.10111>. arXiv:2508.10111.
- Nishanth Sridhar Nakshatri, Shamik Roy, Rajarshi Das, Suthee Chaidaroon, Leonid Boytsov, and Rashmi Gangadharaiyah. Constrained decoding with speculative lookaheads. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 4681–4700, 2025. URL <https://aclanthology.org/2025.naacl-long.239/>.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models, 2025. URL <https://arxiv.org/abs/2502.09992>.

344 Nous Research. json-mode-eval. Hugging Face Datasets, 2024. URL <https://huggingface.co/datasets/NousResearch/json-mode-eval>.
345

346 Kanghee Park, Timothy Zhou, and Loris D’Antoni. Flexible and efficient grammar-constrained
347 decoding. In *International Conference on Machine Learning*, 2025. arXiv:2502.05111.

348 Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and
349 Joseph E. Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic
350 evaluation of large language models. In *International Conference on Machine Learning*, 2025.

351 Gabriel Poesia, Alex Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and
352 Sumit Gulwani. Synchronesh: Reliable code generation from pre-trained language models. In
353 *International Conference on Learning Representations*, 2022.

354 Subham Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin Chiu,
355 Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language
356 models. In *Advances in Neural Information Processing Systems*, volume 37, pages 130136–
357 130184, 2024.

358 Chenze Shao, Zhengrui Ma, and Yang Feng. Viterbi decoding of directed acyclic transformer for non-
359 autoregressive machine translation. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 4390–4397, 2022.
360

361 Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis Titsias. Simplified and generalized
362 masked diffusion for discrete data. In *Advances in Neural Information Processing Systems*,
363 volume 37, pages 103131–103167, 2024.

364 Tarun Suresh, Debangshu Banerjee, Shubham Ugare, Sasa Misailovic, and Gagandeep Singh. Dingo:
365 Constrained inference for diffusion llms, 2025. URL <https://arxiv.org/abs/2505.23061>.

366 Gabriele Tuccio, Luana Bulla, Maria Madonia, Aldo Gangemi, and Misael Mongiovi. GRAMMAR-
367 LLM: Grammar-constrained natural language generation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 3412–3422, 2025. URL <https://aclanthology.org/2025.findings-acl.177/>.
368
369

370 Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. SynCode:
371 LLM generation with grammar augmentation. *Transactions on Machine Learning Research*, 2025.
372 arXiv:2403.01632.

373 Brandon T Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.
374

375 Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng
376 Kong. Dream 7b: Diffusion large language models, 2025. URL <https://arxiv.org/abs/2508.15487>.
377

378 Yitong Zhang, Yongmin Li, Yuetong Liu, Jia Li, Xiaoran Jia, Zherui Li, and Ge Li. Lookahead-
379 then-verify: Reliable constrained decoding for diffusion LLMs under context-free grammars.
380 *Proceedings of the ACM on Software Engineering*, 3(ISSTA), 2026.

381 Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao,
382 Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang:
383 Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024.
384

A The distribution of legal token sets

Section 3.4 rests on a claim about how many tokens an incremental parser actually admits at a live state. This appendix gives the measurement and the full distribution.

Procedure. We take the 40 generated outputs of the DPGrammar run on JSONSchemaBench medium (seed 0, $T=128$) whose schemas compile, and replay each one through `llguidance 1.7.0` under teacher forcing: the parser is initialised from the instance’s schema, and the tokens the model actually produced are fed to it one at a time. Before consuming each token we call `compute_logit_bias()` and count the non-zero entries of the returned mask; that count is $|\mathcal{L}(q)|$, the number of tokens the parser would accept next. Replay stops at the first token the parser rejects, so every state we record is one the decoder genuinely reached. This yields 6,657 live states over a vocabulary of $|V| = 126,349$. The measurement is offline and model-free, which needs only the schema and the emitted text, so it can be reproduced without a GPU.

The distribution. Figure 2 shows the result. The distribution is not merely skewed, it is bimodal: 38.4% of states admit fewer than 100 tokens and 29.7% admit more than 10,000, leaving the decade between 10^4 and 10^5 nearly empty. The median is 106, and the extremes are sharper still: before its first token a JSON object grammar admits 2 of 126,349.

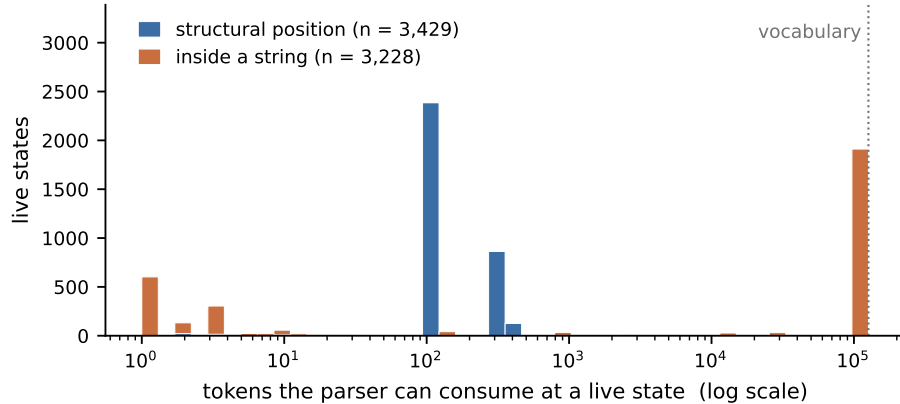


Figure 2: How many tokens an incremental parser admits, over 6,657 live states on JSONSchemaBench medium ($|V| = 126,349$).

What the order of proposal and filter costs. The distribution above is why §3.4 draws candidates from $\mathcal{L}(q)$ rather than ranking V and filtering: at the 38% of live states admitting fewer than 100 tokens, a top-100 draw is aiming at a small target. Reversing the order in our own implementation costs 1.2 pp of schema@1, and dead-ends the search at 93 of its 399 repair calls against 3 of 315 (§4.4). Those 93 are the failure this ordering cannot design away: the draw held no token the parser would accept, so the search reported no path where one existed and handed the position back, 273 positions in all.

One site, both orders. Instance o65372 gives the failure in miniature. At position 17 the model has written " , and the parser refuses it: the schema is spelling out a mandated property name, and from that state it admits *one* token of the 126,349 in the vocabulary, the next character of that name. The model ranks the vocabulary without seeing the grammar, so its top-100 has no particular reason to contain that character, and here it does not. Filtering after sampling therefore leaves nothing: the candidate set is empty at the first layer, the DP dead-ends before it places anything, and position 17 goes back to the sampler. Reading the edge set first leaves the same single token, which is all the search needs; from it the DP repairs a seven-position span in three edits. Nothing about the grammar, the model or the state differs between the two runs. Only the order in which they are consulted does.

Where the two modes come from. The obvious explanation, that the tight mode is structural punctuation and the wide mode is string content, is only half right. Structural positions (3,429 states)

are uniformly tight, with a median of 104 and a quartile range of 95 to 324. The bimodality lives entirely inside strings (3,228 states, $p_{25} = 3$, median = 125,308), where it separates two kinds of string: a free-form value admits almost the whole vocabulary, whereas a partially spelled property name admits a handful of continuations. Both modes matter for repair, and violations do not fall evenly across them. Of the 1,106 the reported run records, 43.9% occur at a state admitting fewer than 100 tokens against 38.4% of live states overall, and 22.4% at one admitting more than 10,000 against 29.7%: the tight mode is over-represented, though only modestly. Its tail is not modest. One violation in nine, 120 of 1,106, arises where the grammar admits a single token, and those are the states a top- k draw over the vocabulary has no reason to hit.

When the automaton gives the complete candidate set. Where $|\mathcal{L}(q)| \leq k$, Eq. 1 enumerates the edge set exactly rather than sampling it, and the candidate set is provably complete:

k	8	32	64	100	128	256
states with $ \mathcal{L}(q) \leq k$	16.4%	17.9%	18.1%	39.7%	54.0%	54.6%

At the reported $k=100$ this holds at 39.7% of live states. The tight mode clusters just above 100, so $k=128$ reaches 54.0% and $k=256$ only 0.6 beyond that: the mode is exhausted by 128. Whether that extra coverage is worth anything is a question we can answer with the same replay used for the merge width. Solving each site twice, at $k=100$ and at $k=128$ and otherwise identically, the two returned the same assignment at every one of 320 solves over 161 states, and not because the budget was idle: 60% of those states admit more than 100 tokens and 31% more than 128. What makes k a safe knob is in any case not its value but the order it is applied in, since a smaller k costs likelihood and cannot cost validity (§3.4). The fraction varies widely across schemas (at $k=32$: median 18.4%, interquartile range 9.8–28.3%, and 100% for the most constrained schema in the sample), so it is a property of the schema rather than a constant of the tokenizer.

The span budget. L bounds the forward scan of §3.2 rather than the candidate set. Over the 315 repair sites of the reported run the scan ends at the first MASK at 75.6% of them and at a depth-zero junction at 9.8%, which leaves the budget itself binding at 7.9%; the remaining 6.7% end where the probe can no longer advance. Spans run to 13.8 positions on average and 9 at the median. A smaller budget would bind more often, at 13.3% of sites for $L=32$, 19.0% for 24, 31.7% for 16 and 52.4% for 8, so $L=48$ sits where that curve has flattened and a span it truncates is already an outlier.

Two facts the method depends on. First, no live state had an empty legal set (0 of 6,657). This is what makes the DP total: a live state always has a continuation, so the candidate set is never empty and the search can always complete the span. Our termination rule (§3.2) treats an empty mask as a failed read rather than as a stop signal for this reason. Second, the mask read is cheap (0.025 ms mean, 0.012 ms median) and the implementation already performs it to compute the state key used for path merging, so drawing candidates from the automaton adds no parser calls beyond those already paid for.

B Ordering artifacts

Ilguidance compiles a JSON schema into a grammar that fixes the order of an object’s properties to their declaration order: $\{ "a" : 1, "b" : 2 \}$ is rejected when the schema declares b first, although both orders satisfy the schema and `jsonschema` accepts either. A model that writes the properties in a different order therefore produces a violation that is not an error, and the correct repair, moving a block, is not expressible by a positional DP, which can only overwrite in place. The repair then writes the demanded key name over a value intended for another field. We measured this class at 6.6% of the 1,106 violations the reported medium run records across 276 instances, and at 22% of the 120 that occur where the grammar admits exactly one token, where a positional DP has no alternative to write.

C System optimisations

The repair layer sits on a frontier-constrained decoder, and the latencies we report depend on how that decoder avoids redundant work. Parser work is not repeated: one incremental parser is carried across denoising steps rather than re-parsing the prefix each time, and the search probes candidates by advancing and rolling back, so a parser is copied only for states that survive. Model work is not repeated either: resampling reuses the logits already in hand instead of a fresh forward pass, and committed tokens are batched: a clean step doubles the batch up to a cap of eight and any violation resets it to one, with the batch held at one until three quarters of the denoising steps have elapsed so that early positions each get a forward pass of context before they are fixed. Finally, mask construction for the next position overlaps the current forward pass, keeping the parser off the critical path.

D Implementation and hyperparameters

Every constant the decoder uses, in one place, so that the method is specified without reading it out of prose. Values are those of the reported runs.

	symbol	value	where it acts
<i>Backbone and schedule</i>			
denoising steps	T	128	
generation length		256 tokens	
block length		32	semi-autoregressive blocks
temperature		0.2	Gumbel noise on the logits
remasking		low-confidence	LLaDA’s reference sampler
seed		0	one seed, see §6
hardware		one A100-80GB per shard	
<i>Repair layer</i>			
candidate budget	k	100	per lattice node (§3.4)
span budget	L	48 positions	forward scan cap (§3.1)
DP position cap		64	never binds while $L < 64$
single-token retry depth		10	ranked logits at v (§3.1)
window policy		full span	one pass, not progressive widening
merge key	κ	$\text{bytes}(\mathcal{L}(q))$	§3.3, App. E
paths kept per key		1	widening it changes nothing measured
<i>Budgets and batching</i>			
hand-backs per instance		100	a run that exceeds it is reported as is
DP wall-clock per instance		240 s	below the 120 s scoring cap
commit batch cap		8 tokens	
commit batch growth		$\times 2$ on a clean step	reset to 1 on any violation
batch growth gated until		$\frac{3}{4}T$	early positions get a pass each
<i>Parser</i>			
library		LLGUIDANCE 1.7.0	App. E
<code>max_items_in_row</code>		20,000	default aborts on deep nesting
<code>step_max_items</code>		600,000	
stop tokens		EOS 126081, EOT 126348	excluded from $\mathcal{C}$
mask token		126336	
vocabulary	$ V $	126,349	from the checkpoint’s tokenizer

Two behaviours the table cannot carry. First, a rejected single-token retry does not count against the hand-back budget: only a commit or a fall-through to the joint search does, so a violation whose repair sits below rank 10 cannot exhaust the budget before the search is reached. Second, mask construction for the next position is issued asynchronously and joined after the current forward pass, and any state change at the frontier discards the pending mask rather than using a stale one.

E The parser interface

The paper treats the incremental parser abstractly: a state q , a consumable set $\mathcal{L}(q)$, and the ability to advance, copy, and undo. This appendix records the concrete interface those abstractions are built on, so that the method can be reimplemented against a different parser.

Construction. We use LLGUIDANCE 1.7.0 [Microsoft, 2025]. A tokenizer is built from the checkpoint’s own `tokenizer.json`, so token boundaries in the grammar and in the model agree exactly. A JSON Schema is compiled with `grammar_from_json_schema` and a SMILES grammar with `grammar_from_lark`; compilation is checked before use, and a schema that fails to compile is excluded from every arm rather than scored as a failure for one (§4 gives the counts). The parser limits are raised above their defaults, at which deeply nested schemas abort mid-parse (Appendix D).

The five operations.

paper	LLGUIDANCE	note
$\mathcal{L}(q)$	<code>compute_logit_bias()</code>	byte mask over V ; 0.025 ms mean
advance	<code>try_consume_tokens([t])</code>	returns tokens consumed; 0 means rejected
copy a state	<code>deep_copy()</code>	lattice nodes hold these
undo	<code>rollback(n)</code>	probe without copying
may stop	<code>is_accepting()</code>	state is final, not that it must stop

Two details the method depends on. First, EOS is marked legal by the bias at accepting states but is not a consumable grammar symbol: `try_consume_tokens([EOS])` returns 0 there. It is therefore excluded from the candidate set of Eq. 1: offering it wastes a slot, and where it is the only token the bias marks legal it would empty the set and imitate a dead end. A termination rule written in terms of consumption would therefore never fire, which is why `only_stop_remains` (§3.2) is defined on the bias, meaning that nothing but stop tokens remains legal, rather than on what the parser will consume. Second, the DP merges lattice paths on `bytes(compute_logit_bias())` as the state key. The key is a value the implementation already computes to build the candidate set, so merging costs no additional parser calls, but it identifies a state by what that state admits next rather than by the parser’s configuration, and the two are not the same thing. An array with `"minItems": 5` gives the separating case, and it is easy to check. Feeding `{"xs": [1, {"xs": [1, 2, {"xs": [1, 2, 3 and {"xs": [1, 2, 3, 4` to the parser leaves four states that owe four, three, two and one more element; all four return byte-identical masks, since all four must write a comma next and none may close the bracket. The mask separates them only once the obligation is discharged: after a fifth element it admits `]` and differs from all of them. Merging on κ therefore collapses states that still differ, keeping one and discarding the rest. §3.3 states what that collapse can and cannot cost.

Collisions are rarer than the definition suggests, because the mask is over whole tokens of a byte-level BPE rather than over grammar symbols: a token spanning several bytes makes the mask reflect what the grammar permits several bytes ahead, so states whose futures diverge within that reach are already separated by their masks. A collision requires the divergence to lie beyond it, as it does above. The key can be refined at no cost, since conjoining `is_accepting()` or the bracket depth the span-end scan already maintains separates states without an additional parser call, and we report the unrefined key because it is what produced the measurements below.

How much the abstraction costs. We measure it by widening the merge rather than by reasoning about it. The DP takes a width B : the best B paths per key survive instead of the best one, so $B=1$ is the reported configuration and $B \rightarrow \infty$ is no merging at all. Widening strictly enlarges the searched set, so a width at which the answer stops moving is a width at which merging costs nothing. We replay the generated outputs of the `medium` run through the parser and, at every structural state reached, outside string literals and with a legal set under 4,000, solve the same span twice, varying only B , with the model’s ranking replaced by random score vectors so that the merge has to survive many orderings of the same candidate sets rather than one. Over 13,513 solves at 4,529 states of 447 instances, $B=2$ changed the assignment at 0.19% and $B=4$ at 0.33%, the second set containing the first; at every one of those the wider search scored strictly higher, by a median of 4.6 nats and at most 21.2. The merge therefore does discard a likelier path, on roughly one solve in three hundred. It discards a *valid* path never, which is the property the layer depends on. Where the lattice is small

532 enough to run with no merging at all ($k=6$ over four positions, 7,836 solves in which the path cap
 533 never bound), the merged search returned the lattice optimum every time, so what the key loses at
 534 $k=100$ is reachable only through candidate sets wider than a handful. For scale, the unmerged arm
 535 carried a median of 1,869 live paths where $B=1$ carried 5.

536 F Prevention on the subset where prevention is available

537 The introduction sets repair against prevention, and the published prevention methods are the ones
 538 we cannot run (§4). Running them would mean compiling each JSON Schema to a finite automaton
 539 first, which the recursive schemas in these splits do not admit at all and which, on the ones that do,
 540 would measure that translation as much as the decoding algorithm; their guarantee is therefore a
 541 property we report rather than a number we measure. So that the contrast is measured rather than
 542 asserted, we implement the prevention side ourselves, on the same backbone, the same prompts and
 543 the same schedule. At each denoising step it conditions the model’s mean-field prediction on the
 544 whole constraint and decodes from the exact constrained posterior

$$p(x^0 \mid x^t, G) \propto \left(\prod_i p_\theta(x_i^0 \mid x^t) \right) \cdot \mathbf{1}[x^0 \in \mathcal{L}(G)],$$

545 which a finite automaton makes tractable: the automaton is a chain over positions, multiplying a
 546 chain by a fully factorised measure leaves a chain, and exact inference is then forward-backward.
 547 Violations cannot occur, so none of the repair machinery is reachable. We report both decoders,
 548 because the distinction matters here: *viterbi* takes the mode of that posterior and *marginal* takes
 549 per-position marginals.

550 The constraint is compiled to an automaton under a nesting bound and an edge budget of 20 million,
 551 which is where a recursive schema is lost, and the arms are scored on 101 instances rather than on the
 552 full split because an A100 minute buys about one instance here. Table 5 reports what the guarantee
 553 costs.

Table 5: Prevention against repair on the first 128 instances of JSONSchemaBench *medium* in identifier order, less 27 on which automaton construction exhausted a 32 GiB container before any decoder ran, leaving 101; same backbone, prompts and schedule throughout. Rates are over all 101, so a schema an arm’s toolchain cannot accept counts against it: the guarantee the prevention arms carry is over the documents they produce, not over the workload. Constraint time is each arm’s own instrumentation and the two harnesses measure it differently, so it is compared as an order of magnitude rather than as a ratio.

Method	constraint built	schema@1 ↑	content ↑	median leaves ↑	mean s ↓
FA, marginal (mass)	66 (65.3%)	59.4	8.9	1	77.5
FA, viterbi (mode)	67 (66.3%)	58.4	18.8	1	57.5
no-DP (ours)	92 (91.1%)	83.2	67.3	13	13.4
DPGrammar	92 (91.1%)	85.1	73.3	13	16.8

554 Prevention pays on all three axes at once. It reaches a third fewer schemas: an automaton is built
 555 for 66% of the sample against the incremental parser’s 91%. The gap is not a tuning choice but the
 556 size of the object being asked for. Of the 34 schemas the construction does not deliver, 16 exceed the
 557 edge budget, at a median of 61.7 million edges and one schema at 2.0 billion against a budget of 20
 558 million, and 16 fail construction outright; the remaining two decode to nothing usable. The recursion
 559 a JSON Schema expresses in a line is what a finite automaton has to pay for in states, and on a
 560 third of this workload it cannot. It is slower by a factor of three to five, and where the repair layer’s
 561 constraint machinery is under a second of a 16.8 s instance, the prevention arms spend 90.5% and
 562 92.4% of wall clock inside inference over the automaton. And on the documents it does produce it is
 563 degenerate: the median prevention output carries a single scalar leaf against DPGrammar’s thirteen,
 564 and only 8.9% of the mass decoder’s and 18.8% of the mode decoder’s outputs clear the five-leaf
 565 floor against 73.3% of ours. This is the failure the floor was introduced to see (§4): the smallest
 566 document a schema admits is both valid and cheap, so a decoder that optimises probability under a
 567 hard constraint is drawn to it, and closing a structure costs one token where content pays entropy at
 568 every position.

Table 6: Grammar coverage is a property of the compiler, not of the decoder. Validity is measured only where both toolchains compile, so the two effects are not confounded.

Method	Toolchain	Schemas compiled	Validity on the shared 351
CD-CFG	RUSTFORMLANG	383/586 (65.4%)	45.0%
LAVE	LLGUIDANCE	511/586 (87.2%)	82.3%
No-DP baseline	LLGUIDANCE	511/586 (87.2%)	92.6%
DPGrammar	LLGUIDANCE	511/586 (87.2%)	97.4%

One number here is worth reading against the guarantee rather than past it. Validity is 58–59% over the sample and 88–91% over the instances the prevention arms actually run, not 100%. The constraint is enforced on every prefix, so nothing invalid is ever *sampled*; what the guarantee does not cover is running out of generation length before the document closes, which leaves a prefix that is in the automaton’s language and not in the schema’s. Prevention removes the violations a decoder commits. It does not remove the ones a budget imposes.

G Comparing on shared populations

CD-CFG [Mündler et al., 2025a] is run from the authors’ released code, which builds its constraint with RUSTFORMLANG, the formal-language library shipped in the same repository, rather than with LLGUIDANCE [Microsoft, 2025]. Counting the schemas that front-end rejects as decoding failures would measure a regex parser rather than a decoding algorithm, so Table 6 reports coverage separately and scores validity only on the 351 medium schemas both toolchains compile. Coverage differs widely: RUSTFORMLANG rejects 203 of 586, mostly over regular-expression quantifiers that Python’s `re` accepts, where LLGUIDANCE accepts 511.

On the shared 351, CD-CFG reaches 45.0% with 19 timeouts against DPGrammar’s 97.4% with none, and every arm scores higher there than on its own full set, so the 160 schemas only LLGUIDANCE compiles are the harder ones. That CD-CFG also falls below the unconstrained decoder at every tier of Table 1 is this coverage gap rather than its decoding. LAVE needs the same treatment for a different reason: it runs on the same 511 we do but reaches the cap on 60, and judging it only where it finishes raises it from 78.3% to 88.7% against our 97.3% on that same 451. Neither adjustment changes the ordering.

H Other decoding schedules

The repair layer fires only after a parser rejection, so it places no requirement on the order in which positions are revealed. That is a claim about applicability, and Table 7 separates it from a claim about the size of the gain. Both arms are rerun end to end under each schedule, so every row is an internally controlled comparison.

Table 7: The same two arms under three decoding schedules, on the 511 medium instances LLGUIDANCE compiles. *sched. end* is the share of instances whose generation stopped because the $T=128$ denoising schedule ran out rather than because the grammar permitted an end.

Schedule	Arm	schema@1	content	rs/inst	violations	DP dead ends	sched. end
low-confidence, block 32	no-DP	91.8	74.6	33.2	—	—	—
	DPG	96.7	84.9	1.2	1,106	3/315	31%
random unmasking order	no-DP	91.6	59.1	63.3	—	—	—
	DPG	92.0	76.3	3.1	2,576	1/733	46%
full parallel, block 256	no-DP	77.3	63.8	30.0	—	—	—
	DPG	92.4	76.9	1.0	872	0/320	39%

Committing the whole sequence in one block rather than in blocks of 32 puts more of it beyond the frontier before the parser reads it, and the validity margin widens from +4.9 to +15.1 pp: the unrepaired baseline falls to 77.3% while the repaired arm holds 92.4%. This is the difficulty gradient of Table 1 arriving along a second axis.

Random order costs both arms, and for a reason the table makes visible. It produces $2.3\times$ the violations of the reported schedule, each of which costs denoising steps to repair, so 46% of instances exhaust the 128-step schedule before the document closes against 31% here. Those truncations are 98% of what DPGrammar fails on under that schedule. The search itself does not fail: it dead-ends once in 733 calls, against three in 315 under the reported schedule. What the fixed step budget costs is validity, not repair.

The content margin survives every schedule and is largest where the schedule is hardest, at +17.2 and +13.1 pp against +10.4 in the reported configuration. The validity margin does not: it ranges from +0.4 to +15.1, tracking how much schedule is left once repair has been paid for. Raising T is the obvious remedy and one we do not test here.

I Where the extra wall time goes

DPGrammar is 3.3 s slower than the unrepaired baseline on medium. Figure 3 attributes that difference to work rather than to a time breakdown, because the instrumented counters cover only about a quarter of wall clock in either arm.

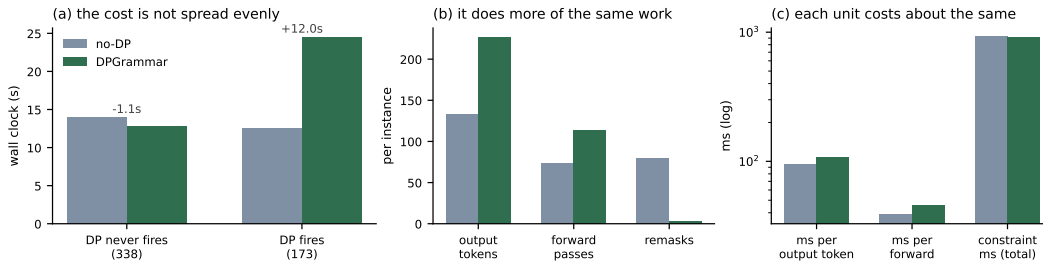


Figure 3: Averages per instance on JSONSchemaBench medium, split by whether the repair layer ever fired. (a) The mean difference is concentrated in the 173 instances where it did; on the other 338 the layer is slightly faster. (b) On those instances it emits 71% more tokens and needs proportionally more forward passes, while remasking $28\times$ less. (c) Per unit the two arms are close, and the constraint machinery costs the same in both.

The bottleneck is not the search. The constraint machinery costs 909 ms against the baseline’s 920, and grammar checks 22.5 ms against 11.2. What changes is how much gets written: the baseline stalls, remasking 79 times and stopping at 133 tokens, while DPGrammar repairs and continues to 227. That takes $1.5\times$ the forward passes, each $1.2\times$ slower on the longer sequence, which is the whole of the difference.

J Why validity alone is not enough

Schema validity can be blind to a difference that matters, which is why §4 pairs it with a content floor rather than reporting it alone. On o78738 the schema asks for a course catalogue entry. Both arms validate, so schema@1 scores them identically; what separates them is that the unrepaired baseline stalls inside prereqs, emits two hundred lines of whitespace, and closes the array empty:

```
no-DP: valid, 4 populated leaves
[ { "code": "MATH 1001", "name": "Calculus I", "credits": "4",
  "description": "An introduction to calculus",
  "requirements": { "prereqs": [
    <200 lines of whitespace>
  ], "coreqs": [] } } ]
```

DPGrammar reaches the same violation at position 120, solves a three-position span in which the parser admits fewer than two tokens on average, and the decoder continues through the rest of the record:

```

DPGrammar: valid, 12 populated leaves
[ { "code": "MATH 1001", "name": "Calculus I", "credits": "4",
  "description": "An introduction to calculus",
  "requirements": {
627   "prereqs": [["MATH 0001", "MATH 0002"]],
    "coreqs": [["MATH 1001", "MATH 1002"]] },
  "lectureHours": "3", "tutorialHours": "-", "labHours": "2",
  "note": "Recommended for first-year students" } ]

```

628 Neither output is wrong about the course; the baseline simply stops saying anything, and validity
 629 cannot see the difference. Only the content floor separates them, and it does so by 8 populated leaves.

630 Neither instance is exceptional: across medium, 18.8% of the baseline's valid outputs carry fewer
 631 than five scalar leaves, against 12.1% of ours.